Handle Control

Feature Description

The Micro:bit is built with an **nRF52833 processor** and equipped with a **2.4GHz RF antenna**, which can be used to realize communication between two Microbits.

Precautions

- Make sure the battery is fully charged.
- This case requires two Microbits and an optional exquisite handle.

Handle Control Principle

Through the radio module of the microbit, different devices can work together through a simple wireless network. When the radio function is enabled on the microbit, it generates a simple wireless local area network. Microbit main boards with the radio function enabled can communicate by setting parameters within an effective range. Wireless communication is divided into two program blocks: sending and receiving. By setting the radio wireless group of both microbit main boards to the same group, they can communicate with each other.

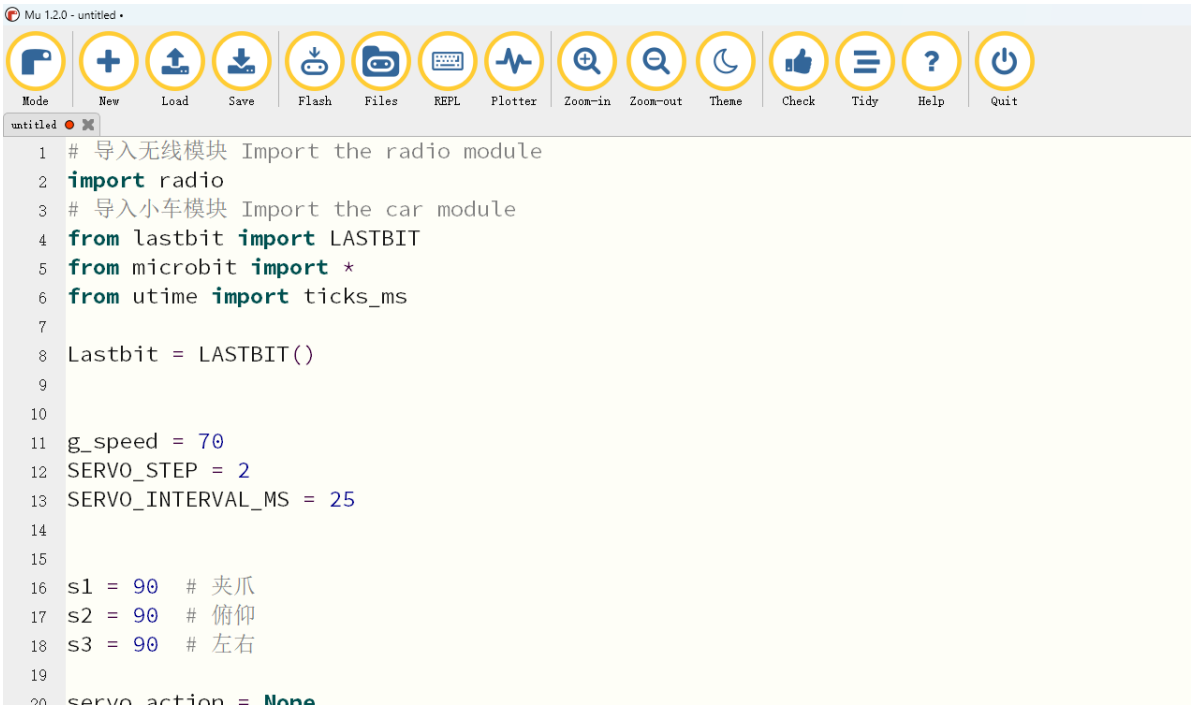
Experimental Steps

Before flashing, make sure both Microbit main boards have been flashed with `LASTBIT.hex`; otherwise, an error will occur and the lastbit control module cannot be imported.

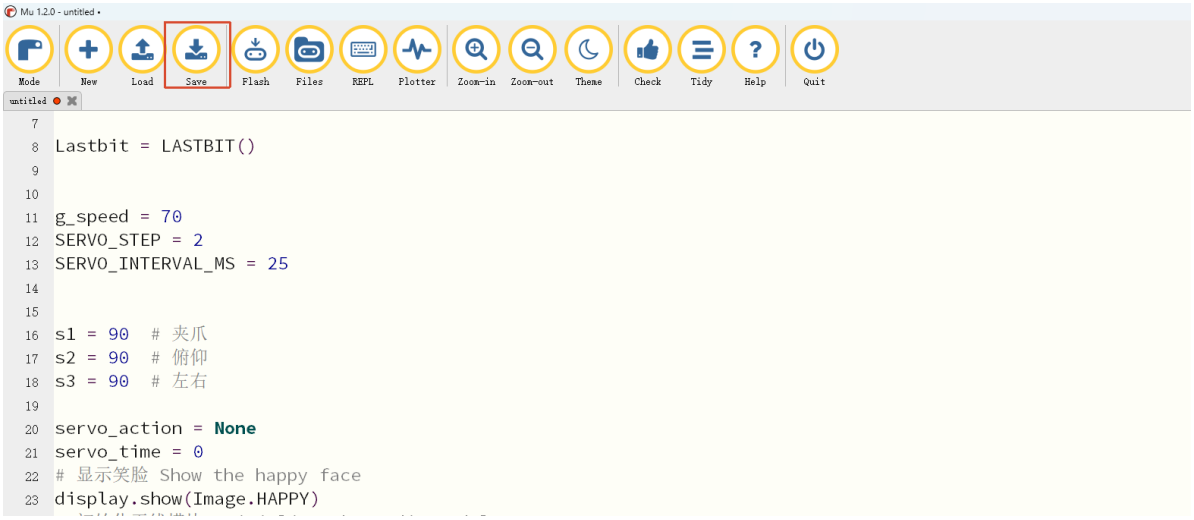
1. Before flashing, switch the switch to the OFF position and connect the computer to the microbit main board using a microUSB cable. **Note: it should be the Microbit interface, not the expansion board's Charging port.**
2. Open the Mu software. Here you can directly copy the program source code we provide; the operation steps are as follows. You can also enter the code yourself in the editor window. Note! All English characters and symbols should be entered in English input mode. Use the Tab key for indentation, and end the last line with a blank line.

   Click the `New` button in the toolbar at the top left, and a file titled untitled will appear.

3. Copy the content of the `Code Implementation` section provided below into the current file. You will notice a small red dot after the title bar, indicating that the code content has been modified but is not yet saved.

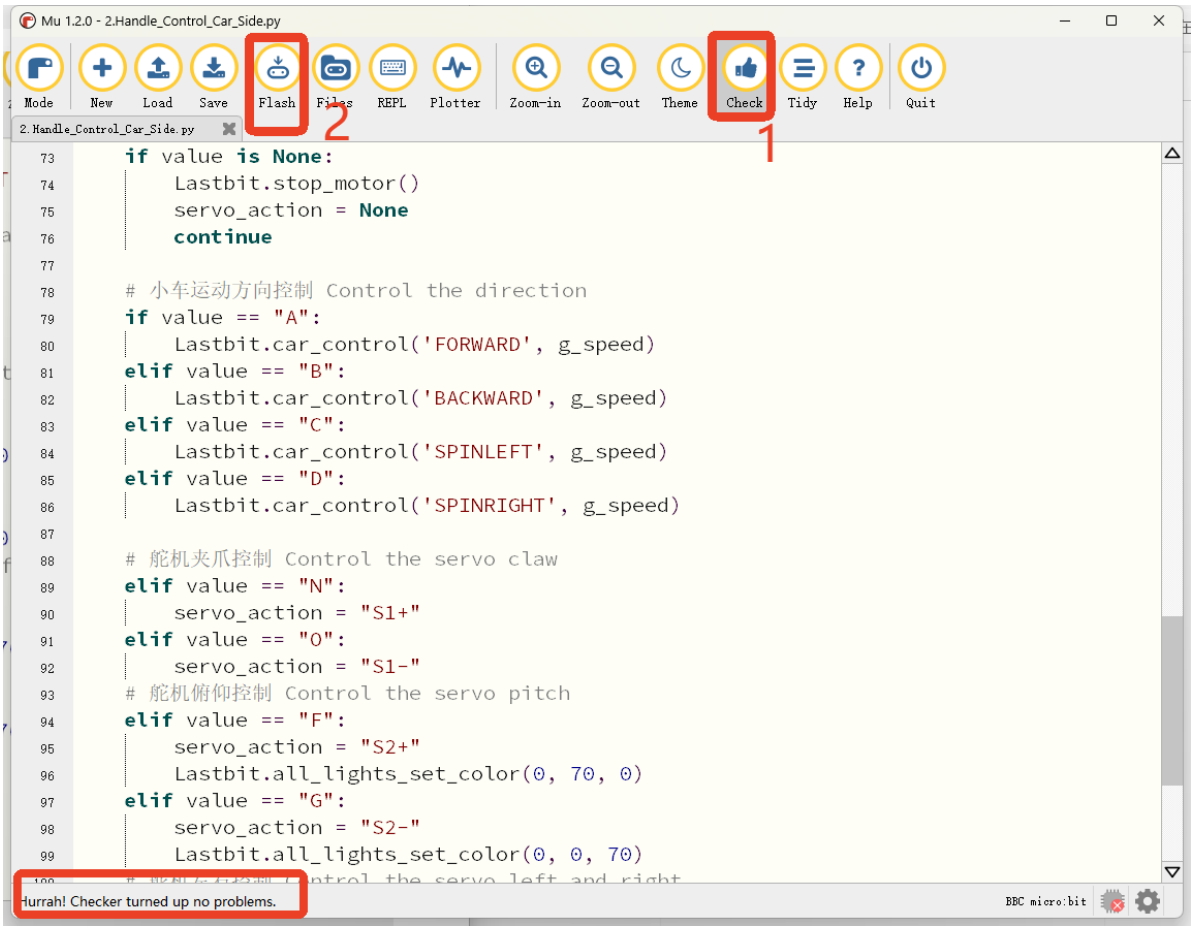


Click the `Save` button in the toolbar above to save the current program to a local file. You can choose the save path yourself; here we save it directly to the desktop. The file is named after `Handle Control Car Side` as an example.



After saving, the red dot in the title bar will disappear. At this point, we can check whether the code has any syntax errors. Click `Check` to check our program, and the editor will display **Awesome! Zero problems found.** at the bottom, meaning no syntax errors were detected. At this point, if you have connected the MICROBIT to the PC in advance following the steps above and have flashed `LASTBIT.hex`, the next step is to download the current program to the Microbit.

Click the `Flash` tool in the toolbar to start flashing.



During flashing, the orange-yellow indicator light near the Microbit main board will blink frequently, and the editor will show `Copied code onto micro:bit.` at the bottom, indicating that the flashing is complete. After flashing, the indicator light will no longer blink.

5. After flashing is complete, unplug the Microusb cable. At this time, the car is in a power-off state. Switch the switch to the (ON) position.
6. Following the same steps, **flash the handle control handle-side program to the Microbit**.

7. Connect the battery to the handle, switch the S2 switch to the ON position, and turn on the handle.

Code Implementation

This case has two source codes that need to be flashed to the corresponding Microbit main boards separately.

Handle Control Car Side

```
#Import the radio module
import radio
# Import the car module
from lastbit import LASTBIT
from microbit import *
from utime import ticks_ms

Lastbit = LASTBIT()


g_speed = 70
SERVO_STEP = 2
SERVO_INTERVAL_MS = 25


s1 = 90  # Gripper
s2 = 90  # Pitch
s3 = 90  # Yaw

servo_action = None
servo_time = 0
# Show the happy face
display.show(Image.HAPPY)
# Initialize the radio module
radio.on()
#  Set the radio group number
radio.config(group=1)
#  Initialize the servo
Lastbit.set_servo('S1', s1)
Lastbit.set_servo('S2', s2)
Lastbit.set_servo('S3', s3)
sleep(1000)
#  Show the "T"
display.show(Image("99999:"
                   "00900:"
                   "00900:"
                   "00900:"
                   "00900"))

# Control the servo
def servo_control():
    global servo_time, s1, s2, s3

    now = ticks_ms()
    if now - servo_time < SERVO_INTERVAL_MS:
        return
    servo_time = now
```

```
    if servo_action == "S1+":
        s1 = min(165, s1 + SERVO_STEP)
        Lastbit.set_servo('S1', s1)
    elif servo_action == "S1-":
        s1 = max(60, s1 - SERVO_STEP)
        Lastbit.set_servo('S1', s1)

    elif servo_action == "S2+":
        s2 = min(150, s2 + SERVO_STEP)
        Lastbit.set_servo('S2', s2)
    elif servo_action == "S2-":
        s2 = max(60, s2 - SERVO_STEP)
        Lastbit.set_servo('S2', s2)

    elif servo_action == "S3+":
        s3 = min(170, s3 + SERVO_STEP)
        Lastbit.set_servo('S3', s3)
    elif servo_action == "S3-":
        s3 = max(20, s3 - SERVO_STEP)
        Lastbit.set_servo('S3', s3)


while True:
    #  Receive the radio signal
    value = radio.receive()
    now = ticks_ms()

   #  If no signal is received, stop the car
    if value is None:
        Lastbit.stop_motor()
        servo_action = None
        continue

    #  Control the direction
    if value == "A":
        Lastbit.car_control('FORWARD', g_speed)
    elif value == "B":
        Lastbit.car_control('BACKWARD', g_speed)
    elif value == "C":
        Lastbit.car_control('SPINLEFT', g_speed)
    elif value == "D":
        Lastbit.car_control('SPINRIGHT', g_speed)

    # Control the servo claw
    elif value == "N":
        servo_action = "S1+"
    elif value == "O":
        servo_action = "S1-"
    #  Control the servo pitch
    elif value == "F":
        servo_action = "S2+"
        Lastbit.all_lights_set_color(0, 70, 0)
    elif value == "G":
        servo_action = "S2-"
        Lastbit.all_lights_set_color(0, 0, 70)
    #  Control the servo left and right
    elif value == "E":
```

```
        servo_action = "S3+"
        Lastbit.all_lights_set_color(70, 0, 0)
    elif value == "H":
        servo_action = "S3-"
        Lastbit.all_lights_set_color(70, 70, 0)
    #  Press the joystick
    elif value == "I":
        Lastbit.all_lights_off()


    servo_control()
```

Handle Control Handle Side

```
# Controller side of the handle control
from microbit import *
#  Import the hand control control module
import ghandle
# Import the radio module
import radio

display.show(Image.HEART)
#  Initialize the radio module
radio.on()
# Set the radio group number
radio.config(group=1)

while True:
    #  Control the joystick
    if ghandle.rocker(ghandle.up):
        radio.send('A')
        display.show(Image.ARROW_N)
    elif ghandle.rocker(ghandle.down):
        radio.send('B')
        display.show(Image.ARROW_S)
    elif ghandle.rocker(ghandle.left):
        radio.send('C')
        display.show(Image.ARROW_W)
    elif ghandle.rocker(ghandle.right):
        radio.send('D')
        display.show(Image.ARROW_E)
    #  Press the joystick
    elif ghandle.rocker(ghandle.pressed):
        radio.send('I')
        display.show(Image.NO)
    elif not (ghandle.rocker(ghandle.up) or ghandle.rocker(ghandle.down) or
              ghandle.rocker(ghandle.left) or ghandle.rocker(ghandle.right) or
              ghandle.rocker(ghandle.pressed)):
        radio.send('O')
        display.clear()
    #  Control the servo
    if ghandle.B1_is_pressed():
        radio.send('E')

    if ghandle.B2_is_pressed():
        radio.send('F')
```

```
    if ghandle.B3_is_pressed():
        radio.send('G')

    if ghandle.B4_is_pressed():
        radio.send('H')
    # # Controlled by A/B buttons
    # Control the claw
    if button_a.is_pressed():
        radio.send('N')
    elif button_b.is_pressed():
        radio.send('O')
```

Experimental Phenomenon

After turning on the handle, the car will perform different actions when different keys are pressed.

Key Description	Function Description
Joystick Up	The car moves forward
Joystick Down	The car moves backward
Joystick Left	The car turns left
Joystick Right	The car turns right
Joystick Pressed	Turn off the lights
B2 (Green)	Turn on the green light, the robotic arm moves up
B1 (Red)	Turn on the red light, the robotic arm moves left
B3 (Blue)	Turn on the blue light, the robotic arm moves down
B4 (Yellow)	Turn on the yellow light, the robotic arm moves right
A Button	Control the robotic arm gripper to tighten
B Button	Control the robotic arm gripper to release
S2 (OFF|ON)	Handle power switch; switch to (ON) when in use
S3 (Software|Button)	Vibration switch (when switched to the Button side, pressing the right button will cause vibration and the buzzer will sound)